

Learning the Pattern, Not the Noise

What overfitting is, where it comes from, and four ways to hold a model back: L2, L1, early stopping and dropout. Small numbers by hand, then a live model to watch.

How to use this sheet. Read one section at a time. Type your answers into the boxes and press **Check**; open *Show answer* only after that. Each part ends with a lab: a two-class model that trains in your browser while you watch its decision boundary and its training and validation curves. Nothing plays until you press **Train**.

PART 1

What overfitting really is

A model is not built to score well on data it has already seen. It is built to work on data it has never seen.

1.1 What we actually want

Two students prepare for an exam with past papers. One **memorises** the answers to every past question. The other **learns the method** behind the questions. On the past papers both score 100%. On the real exam, only the second does well.

- **Training data** is the past papers.
- **Test data** is the real exam.
- **Generalisation** is doing well on the real exam.

KEY IDEA

A low training error is not the goal. A low error on **new** data is.

1.2 Signal and noise

Real data is always **pattern + noise**. Two houses of the same size sell for different prices because of things we did not measure. The model cannot tell which part of a point is pattern and which is noise. It only sees the points.

A model that fits every point exactly has **learned the noise too**, and the noise will not repeat on new data.

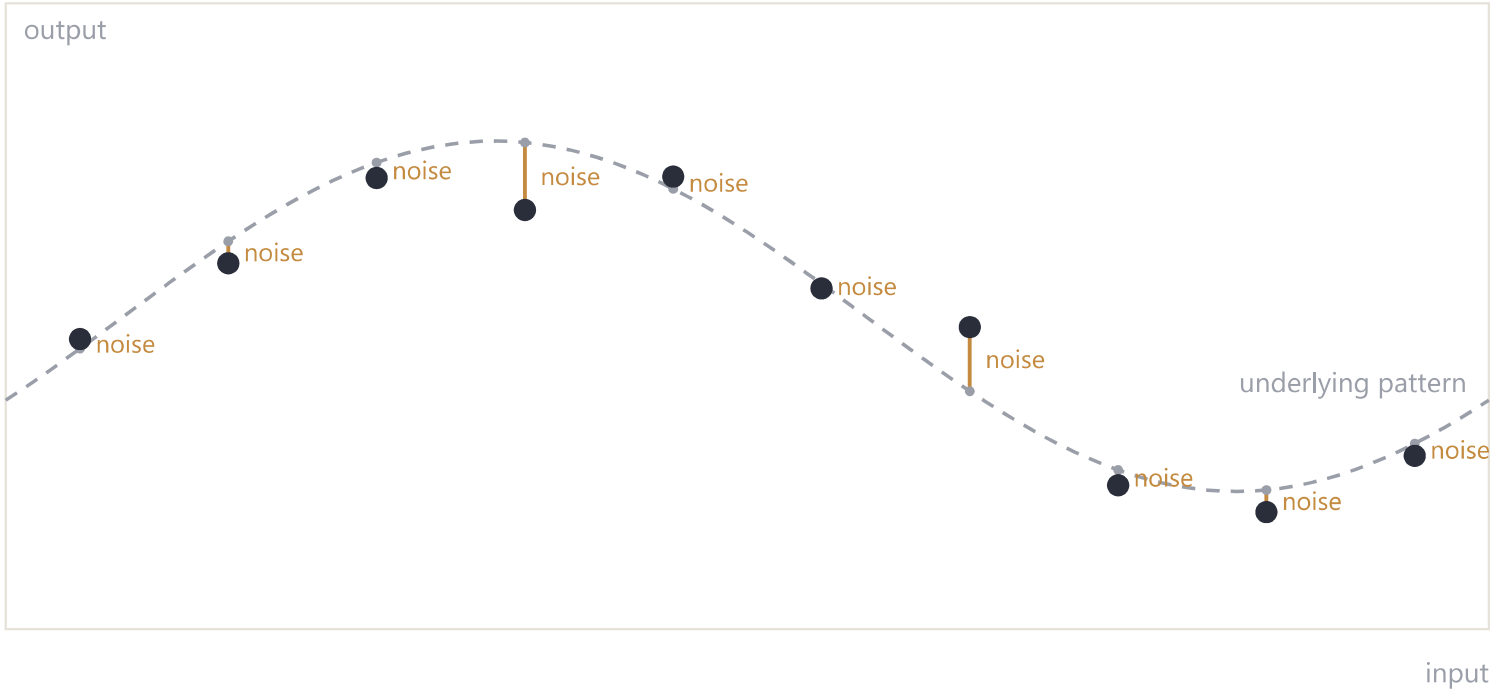
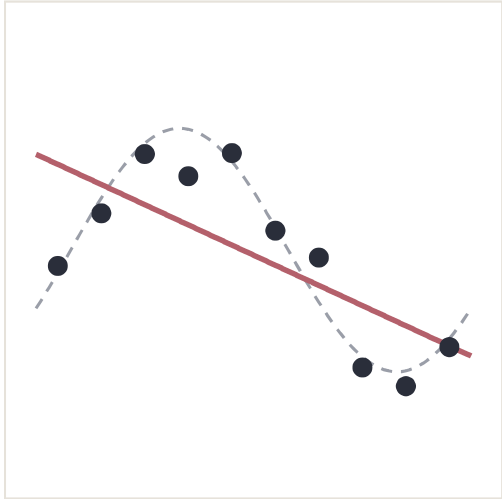
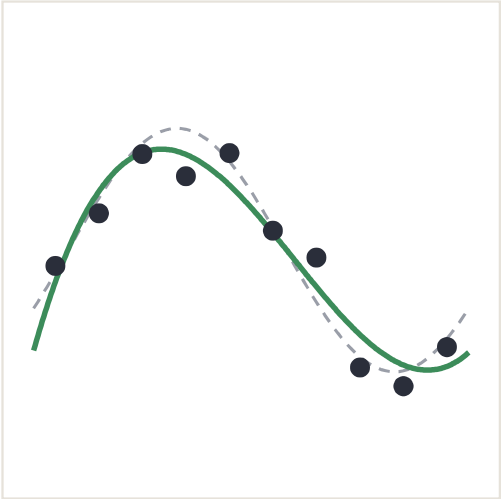


Figure 1.2 Ten measurements of a smooth underlying pattern (dashed). Each point sits a little above or below it. The amber distance is that point's noise.

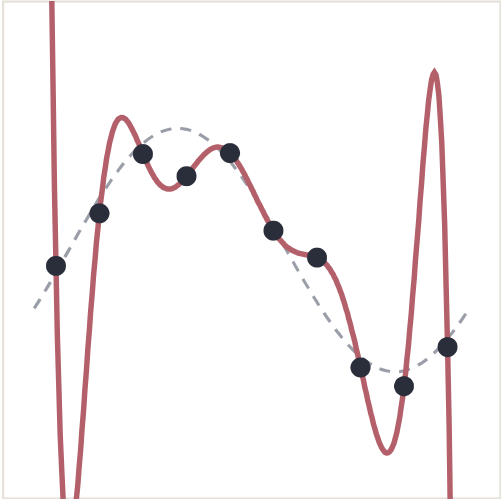
1.3 Three models, same data



Underfit
straight line



Good fit
gentle curve



Overfit
through every point

Figure 1.3 The same ten points fitted by a straight line, a gentle curve and a curve with enough freedom to pass through every point. Only the middle one is close to the dashed pattern.

TRY IT BY HAND

Squared error = (actual – predicted)². **MSE** is the average of the squared errors.

	Training set				Test set	
Actual	5	7	6	8	6	9
Model A (smooth) predicts	6	6	7	7	7	8
Model B (wiggly) predicts	5	7	6	8	9	6

- a) Training MSE: A = B =
- b) Test MSE: A = B =
- c) Which model would you deploy?

1.4 The same story in classification

In classification the model draws a **decision boundary**: the line that separates the **blue** ○ class from the **red** × class. A stray point, a red × sitting among the blue ○s, is noise. An overfit model draws an **island** around it, and new blue ○ points that land inside the island are then classified wrongly.

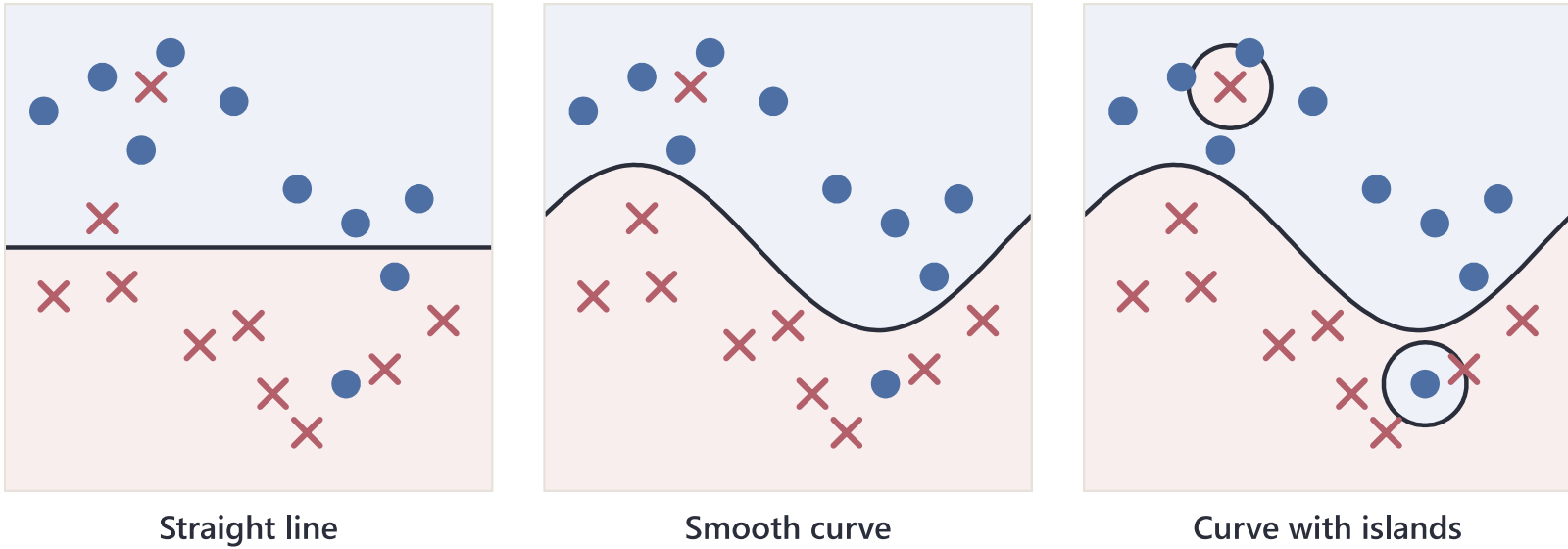


Figure 1.4 The same twenty training points three times. Left: a straight line. Middle: a smooth curve. Right: the same curve plus an island around each stray point. The islands fix two training mistakes and will cause new ones.

KEY IDEA

Overfitting in classification looks like **islands and sharp bends** around individual points.

1.5 How overfitting shows up in the curves

During training we measure the loss on the training set and on a separate **validation set** after every epoch. The space between the two curves is the **generalisation gap**.

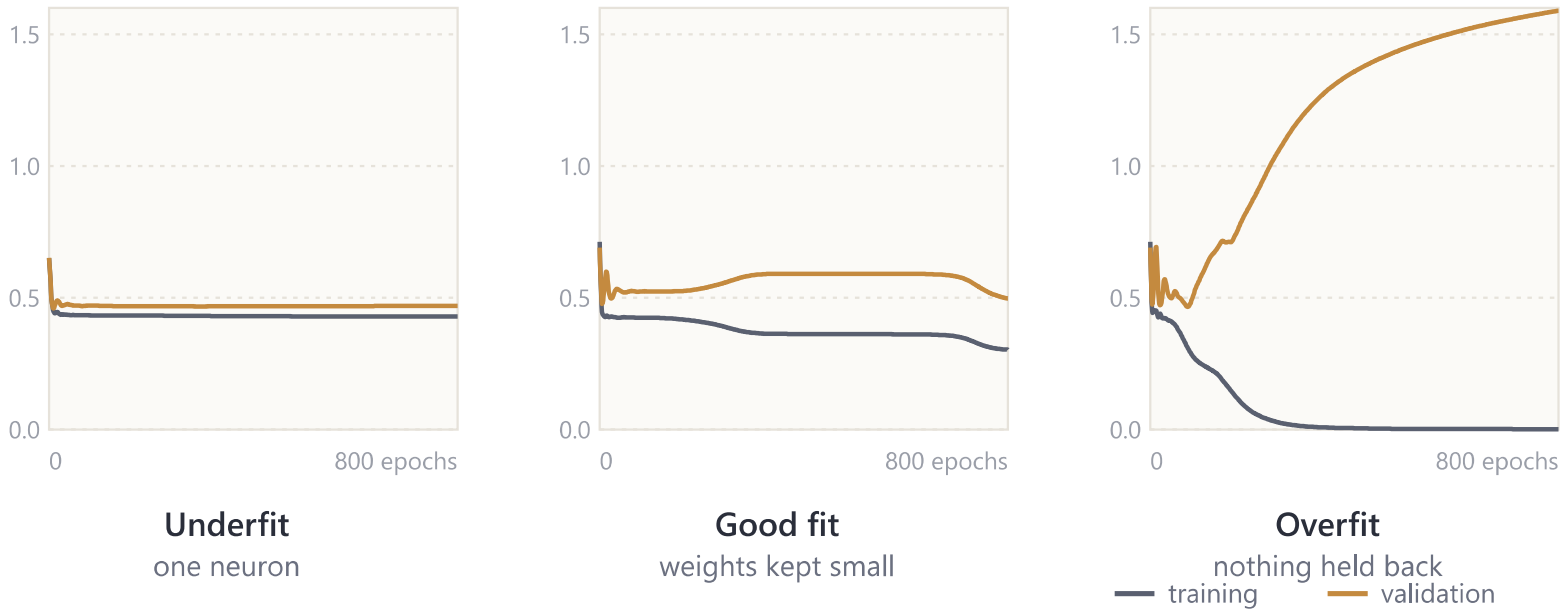


Figure 1.5 Three real runs on the lab data of Part 1. Left: a one-neuron network. Middle: the 50-50 network with its weights kept small. Right: the same 50-50 network left free. Grey is training loss, amber is validation loss.

- **Underfit:** both curves stay high and close together.
- **Good fit:** both curves are low, with a small gap.
- **Overfit:** training loss keeps falling. Validation loss falls, reaches a lowest point, then rises.

CHECK

Match each curve panel to a boundary panel in Figure 1.4: left →
middle → right →

COMMON MISTAKE

“Training loss is still falling, so the model is still improving.” Watch the **validation** curve instead.

1.6 Why overfitting happens: three ingredients

- 1. **Too much capacity.** The model has far more freedom than the data needs.
- 2. **Too little data.** Few points to pin the pattern down.
- 3. **Training too long.** Once the pattern is learned, the model starts chasing the noise.

TRY IT BY HAND

The lab network has 2 inputs, then 50 neurons, then 50 neurons, then 1 output. Each layer has (inputs × outputs) weights plus one bias per output.

Layer 1: $2 \times 50 + 50 =$

Layer 2: $50 \times 50 + 50 =$

Layer 3: $50 \times 1 + 1 =$

Total = . The lab trains on 40 points. Parameters per training point $\approx$

1.7 Where does overfitting come from? Big weights

A neuron computes $z = w \cdot x$ and passes it through a sigmoid. The size of w decides how sharply the output changes.

Sigmoid values you will need: $\sigma(-5) = 0.007$, $\sigma(-0.1) = 0.475$, $\sigma(0) = 0.5$, $\sigma(0.1) = 0.525$, $\sigma(5) = 0.993$.

TRY IT BY HAND

Find the output $\sigma(w \cdot x)$ at three nearby inputs.

	$x = -0.1$	$x = 0$	$x = 0.1$
$w = 1$			
$w = 50$			

Check

Show answer

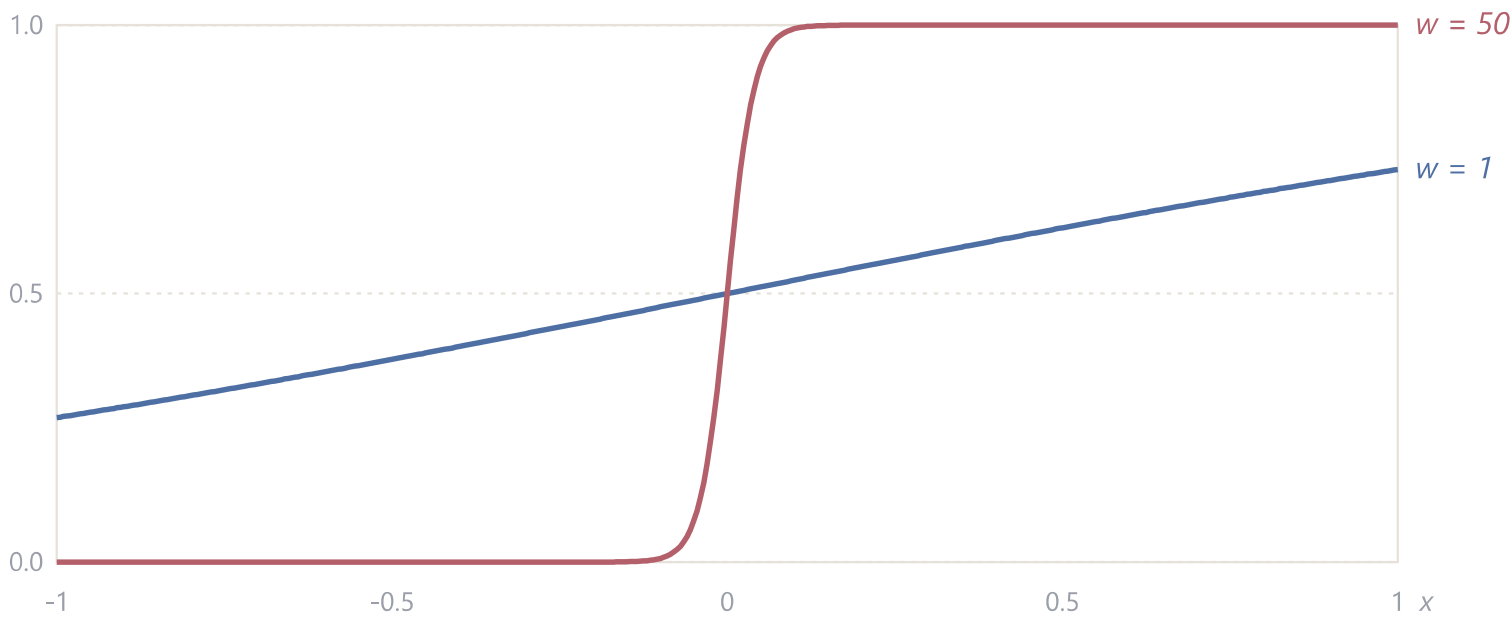


Figure 1.7 $\sigma(w \cdot x)$ for $w = 1$ and $w = 50$. A big weight turns a smooth slope into a cliff.

KEY IDEA

Sharp bends and islands **need big weights**. Small weights can only draw gentle, smooth boundaries. This one idea explains most of what follows.

1.8 What “regularisation” means

Regularisation is any change that makes a model prefer a simpler explanation, even if it fits the training data slightly worse. This sheet builds four of them.

Method	The idea in one line
L2	Charge a price for big weights
L1	Charge a price for every weight, so useless ones become zero
Early stopping	Stop before the model starts memorising
Dropout	Randomly switch neurons off, so memorising becomes hard

LAB 1 Watch it overfit

Model: 2 inputs → 50 tanh → 50 tanh → 1 sigmoid, 2 751 parameters. **Data:** 40 training points, 4 with a wrong label; 200 validation points. **Training:** gradient descent with momentum, learning rate 0.3, 800 epochs, nothing held back. The hollow marker follows the lowest validation loss so far.

```
model = Sequential([
    Dense(50, activation="tanh"),    # 2 inputs -> 50
    Dense(50, activation="tanh"),    # 50 -> 50
    Dense(1, activation="sigmoid"),  # 50 -> 1
])
```

Train

Reset

SPEED

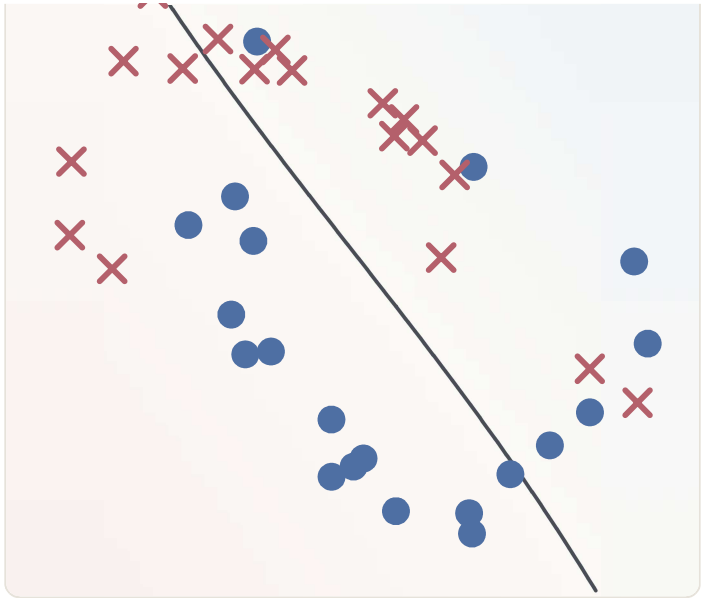
slow

normal

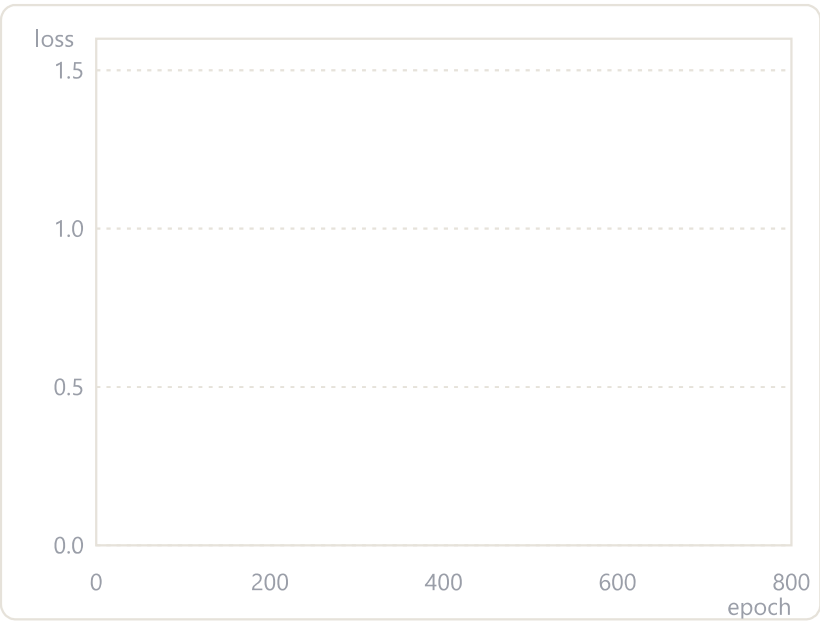
fast

DECISION BOUNDARY

LOSS PER EPOCH



● class 0 × class 1 □ validation points



— training loss — validation loss ○ lowest validation loss

EPOCH	TRAIN LOSS	TRAIN ACC	VAL LOSS	VAL ACC	LARGEST W
0	—	—	—	—	0.34

WHAT TO NOTICE

- The boundary starts almost straight, bends to follow the two arcs, then wraps **islands** around single points.
- Training loss keeps falling. Validation loss turns up after about epoch 75, and the hollow marker stays at that lowest point.
- The largest weight grows past 5. Islands need big weights (1.7).

PART 2

L2 regularisation

Until now the model only cared about the data loss. Now every weight has a price.

2.1 Weights cost something

Total loss = Data loss + **Penalty**

Think of it as a budget. The model may keep a big weight only if that weight earns back more, in lower data loss, than it costs.

2.2 The L2 penalty

$$\text{Penalty} = \frac{\lambda}{2} (w_1^2 + w_2^2 + \dots)$$

- λ (lambda) is how expensive weights are. You choose it.
- Squaring means big weights pay much more. A weight of 1 costs 1; a weight of 10 costs 100.

TRY IT BY HAND

$\lambda = 0.1$, so $\lambda/2 = 0.05$.

Weights [3, −1, 2]: sum of squares = , penalty =

Weights [30, −10, 20]: sum of squares = , penalty =

Keras writes the penalty as $\lambda \Sigma w^2$, without the $\frac{1}{2}$. That changes the number you type, not the idea.

2.3 The update rule, step by step

Plain gradient descent. α is the learning rate.

1
$$w_{\text{new}} = w - \alpha \frac{\partial L}{\partial w}$$

Add the penalty to the loss.

2
$$L_{\text{total}} = L + \frac{\lambda}{2} w^2$$

Differentiate. The gradient of $(\lambda/2)w^2$ is λw , so a new term appears.

3
$$\frac{\partial L_{\text{total}}}{\partial w} = \frac{\partial L}{\partial w} + \lambda w$$

Put it into the update rule.

4
$$w_{\text{new}} = w - \alpha \left(\frac{\partial L}{\partial w} + \lambda w \right)$$

Rearrange: collect the two w terms.

5
$$w_{\text{new}} = (1 - \alpha\lambda) \cdot w - \alpha \frac{\partial L}{\partial w}$$

The step now has two parts:

- 1. **Shrink** the weight a little: multiply by $(1 - \alpha\lambda)$.
- 2. Take the **usual step**: subtract $\alpha \partial L/\partial w$.

TRY IT BY HAND

$w = 2, \partial L/\partial w = 0.5, \alpha = 0.1, \lambda = 1$.

Shrink factor $(1 - \alpha\lambda) =$

w_{new} with L2 =

w_{new} without L2 =

2.4 Why it is called “weight decay”

Suppose a weight is **useless**: its data gradient $\partial L/\partial w$ is 0. With a shrink factor of 0.9 it fades step by step.

TRY IT BY HAND

2 → → →

Check

Show answer

A **useful** weight survives because its data gradient keeps pushing back. It settles where the push from the data balances the shrink.

TRY IT BY HAND

The data loss is $\frac{1}{2}(w - 4)^2$, so without a penalty the best weight is 4. With L2 the best weight becomes $w = 4 / (1 + \lambda)$.

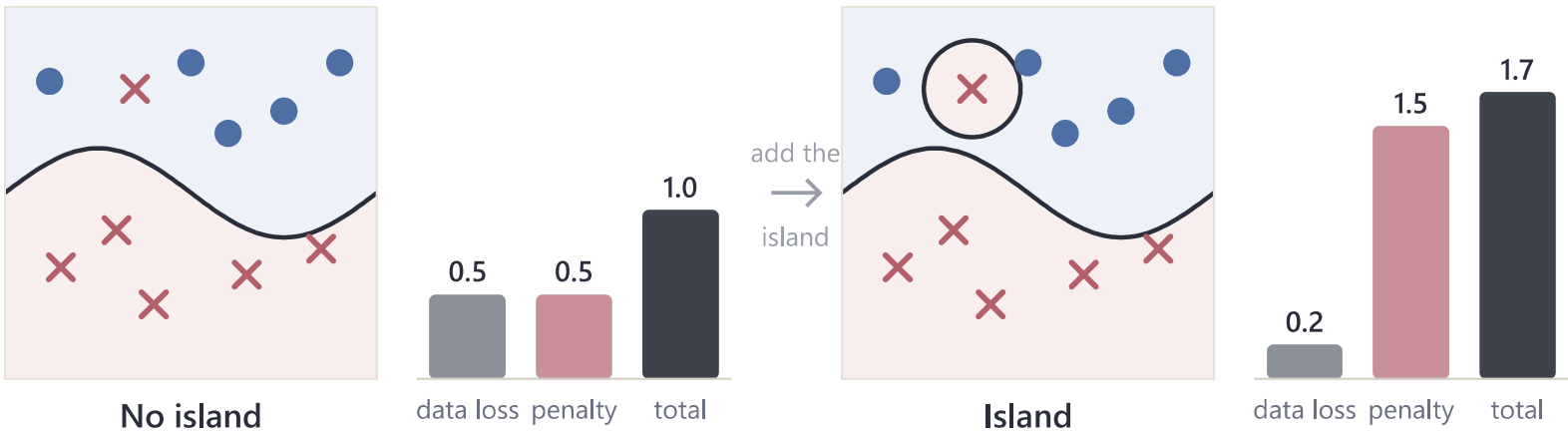
λ	0	1	3
best w			

Check

Show answer

2.5 Why L2 gives a smoother boundary

Go back to 1.7: an island around one stray point needs big weights. L2 asks whether the island is worth its cost.



data loss – 0.3, penalty + 1.0, total + 0.7

$\lambda = 0.1$ · the total is what the model minimises, and the island makes it larger

Figure 2.5 The same stray point with and without an island, $\lambda = 0.1$. The island lowers the data loss by 0.3 but its big weights raise the penalty by 1.0. The total goes up, so a model trained with L2 does not draw it.

TRY IT BY HAND

Drawing the island lowers the data loss by 0.3 and raises the sum of squared weights by 20.

λ	Extra penalty = $(\lambda/2) \times 20$	Island worth it?
0.01		
0.1		

At what λ is the trade exactly break-even?

Check

Show answer

KEY IDEA

L2 does not forbid islands. It makes the model **pay** for them. An island around a single noisy point rarely pays for itself. A real pattern supported by many points does, so it stays.

2.6 Two practical rules

Do not penalise biases. A bias only **shifts** the boundary; it cannot bend it. There are also very few biases.

Scale your features first. Suppose house size (around 100 m²) and number of rooms (around 3) matter equally, each adding about 30 to the output.

TRY IT BY HAND

$w_{\text{size}} =$ (because $w \times 100 = 30$). $w_{\text{rooms}} =$ (because $w \times 3 = 30$).
Their L2 costs, w^2 : and

Check

Show answer

2.7 A note on Adam

With plain gradient descent, “add L2 to the loss” and “shrink the weights each step” are the same thing (2.3). With **Adam** they are not: Adam rescales every gradient, and if the penalty sits inside the loss it gets rescaled too, so equal weights no longer shrink equally. The fix is **AdamW**, which takes the Adam step from the data gradient only and shrinks the weights separately.

COMMON MISTAKE

Both of these put the penalty **inside the loss**. Use AdamW for true weight decay.

PYTORCH

```
# L2 inside the loss (rescaled by Adam)
optim.Adam(params, lr=1e-3, weight_decay=1e-4)

# true weight decay
optim.AdamW(params, lr=1e-3, weight_decay=1e-4)
```

KERAS

```
# L2 inside the loss (rescaled by Adam)
Dense(50, kernel_regularizer=l2(1e-4))

# true weight decay
optimizers.AdamW(learning_rate=1e-3,
                  weight_decay=1e-2)
```

The lab uses plain gradient descent with momentum, so its λ means exactly what 2.3 says.

LAB 2 Charge for big weights

Same model, same data, with the L2 shrink of 2.3 applied to every weight and no bias. Pick λ , then Train.

L2 λ

0

0.003

0.01

0.03

0.1

0.3

Train

Reset

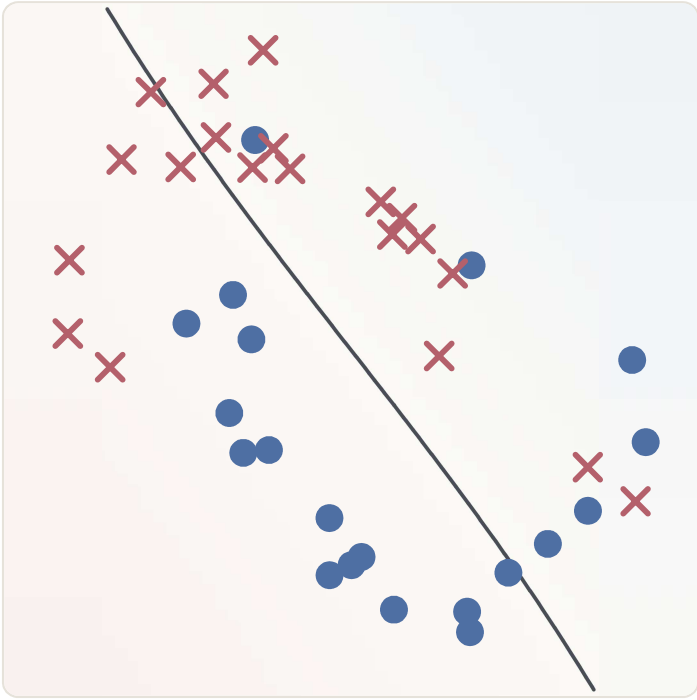
SPEED

slow

normal

fast

DECISION BOUNDARY

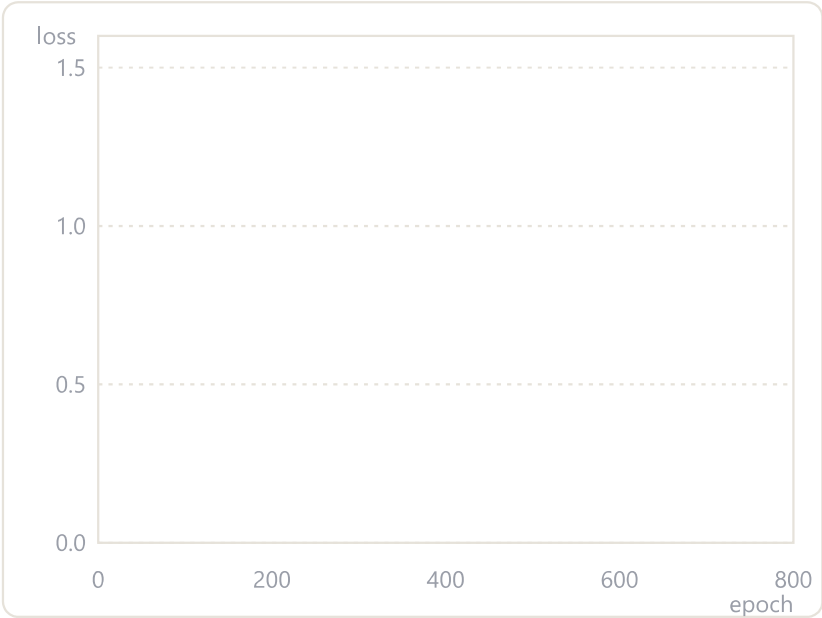


● class O

✕ class x

□ validation points

LOSS PER EPOCH



— training loss

— validation loss

○ lowest validation loss

EPOCH	TRAIN LOSS	TRAIN ACC	VAL LOSS	VAL ACC	LARGEST W
0	—	—	—	—	0.34

WHAT TO NOTICE

- $\lambda = 0.03$ keeps the two arcs and no island closes, though the boundary still bends towards the mislabelled points. The validation curve rises far more slowly.
- $\lambda = 0.3$ flattens the boundary to nearly a line, and the training curve stays high: **underfitting**.
- The largest weight tracks λ : the stronger the penalty, the smaller it stays.
- Try values on a log scale (0.003, 0.01, 0.03, 0.1) and choose with the **validation** readout, never the test score.

PART 3

L1 regularisation

A different price, and one very different consequence: weights that become exactly zero.

3.1 A different price

Penalty = $\lambda (|w_1| + |w_2| + \dots)$

WORKED EXAMPLE

The two costs side by side, leaving λ out.

w	0.1	1	10
-----	-----	---	----

https://kartikgupta98.github.io/dl-worksheets/worksheet09_regularisation.html#p7

10/24

L2 cost w^2	0.01	1	100
L1 cost $ w $	0.1	1	10
L2 barely charges for small weights and charges heavily for big ones. L1 charges every unit of weight the same, including the small ones.			

3.2 The update rule, step by step

Same flow as 2.3. The only change is the penalty.

Plain gradient descent.

1
$$w_{\text{new}} = w - \alpha \frac{\partial L}{\partial w}$$

Add the L1 penalty to the loss.

2
$$L_{\text{total}} = L + \lambda |w|$$

Differentiate. The slope of $|w|$ is +1 for positive w and -1 for negative w : **sign(w)**.

3
$$\frac{\partial L_{\text{total}}}{\partial w} = \frac{\partial L}{\partial w} + \lambda \text{sign}(w)$$

Put it into the update rule.

4
$$w_{\text{new}} = w - \alpha \frac{\partial L}{\partial w} - \alpha \lambda \text{sign}(w)$$

KEY IDEA

The new term does not depend on the size of w . Every weight moves the **same fixed amount**, $\alpha \lambda$, towards zero at every step. L2 shrinks in proportion; L1 subtracts a constant.

3.3 The push towards zero, L2 against L1

- **L2 push** = λw . It gets **weaker** as w shrinks.
- **L1 push** = $\lambda \text{sign}(w)$. It **stays the same size** all the way to zero.

WORKED EXAMPLE

$\lambda = 0.5$.

w	4	2	1	0.5	0.1
L2 push λw	2	1	0.5	0.25	0.05
L1 push λ	0.5	0.5	0.5	0.5	0.5

3.4 Walk it to zero

A useless weight ($\partial L / \partial w = 0$) starts at $w = 1$, with $\alpha = 0.1$ and $\lambda = 2.5$, so $\alpha \lambda = 0.25$.

- **L2:** multiply by 0.75 each step.
- **L1:** subtract 0.25 each step, and stop at 0.

WORKED EXAMPLE

Step	0	1	2	3	4
L2	1	0.75	0.56	0.42	0.32
L1	1	0.75	0.50	0.25	0

L2 **never reaches zero**. L1 reaches it in four steps and stays.

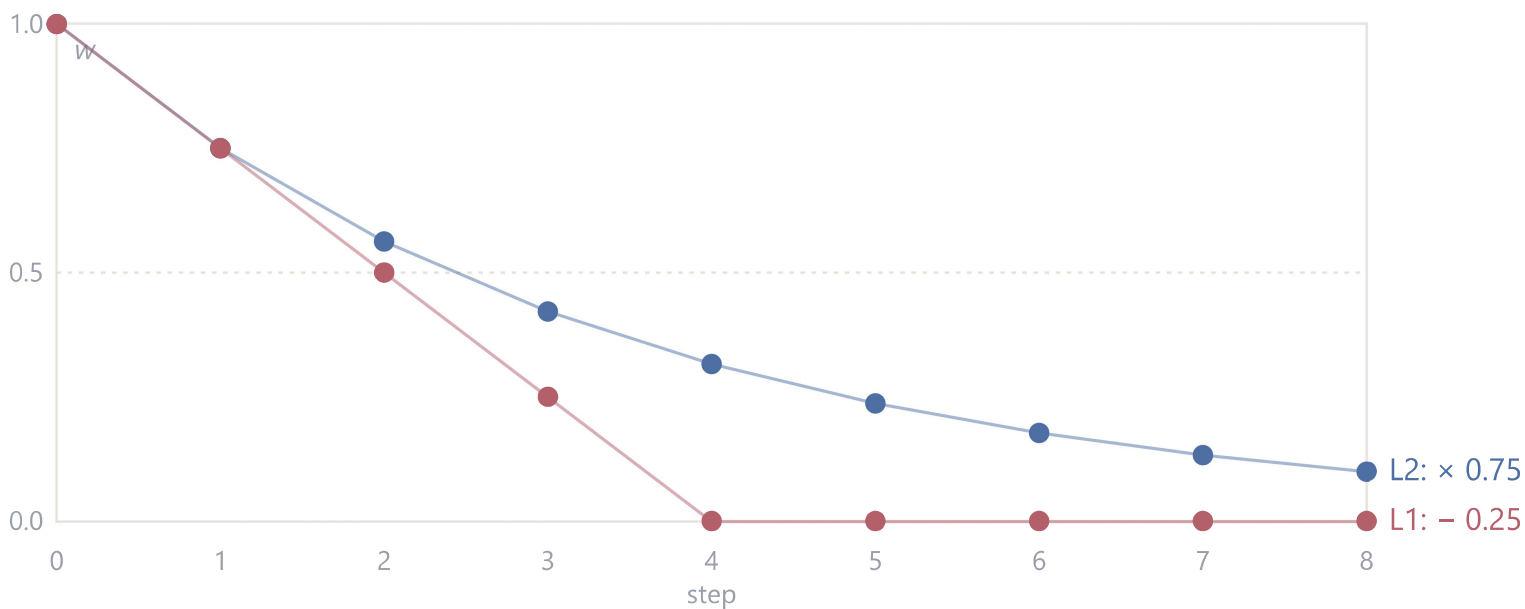


Figure 3.4 The two sequences continued for eight steps. L2 flattens out above zero. L1 reaches it and stays.

KEY IDEA

- L2 divides every weight. All get smaller, none becomes zero.
- L1 subtracts a fixed amount. Weak weights become **exactly zero**. Strong weights lose less than under L2.

3.5 What that buys: automatic feature selection

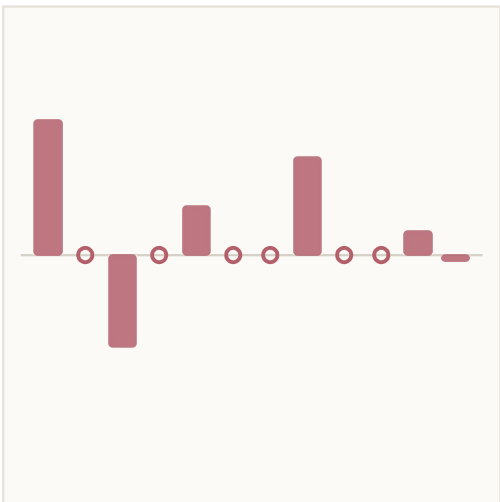
If an input is useless, L1 drives its weights to **exactly zero** and the model ignores that input. L2 leaves every weight small but alive.



No penalty



With L2



With L1
irrelevant: exactly zero

Figure 3.5 Twelve weights after training. Left: with no penalty. Middle: with L2, all twelve shrunk. Right: with L1, six are exactly zero. Schematic.

3.6 When to use L2, when to use L1

L2 is the default in deep learning.

- Hidden neurons are not meaningful features. Deep networks spread a pattern across many weights, each contributing a little. L2 keeps all of them alive and small, which is exactly that regime.
- Its effect changes **gradually** with λ , so it is easy to tune.
- It is what “weight decay” means in every framework (with AdamW, 2.7).

L1 is a tool for specific jobs.

- **Feature selection** on the first layer, when you suspect many inputs are useless.
- **Compression**: a sparse model with most weights at zero is smaller and faster.
- Its effect has a **cliff**: a λ that zeroes 90% of the weights may still overfit, and a slightly larger one may zero almost everything. Harder to tune.
- With plain gradient steps a weight can jump back and forth across zero. Libraries handle this; the lab clips at zero.
- **Elastic net** is L1 and L2 together, when you want a little of each.

LAB 3 Subtract and clip

Same model with L1: after each step every weight moves a fixed amount $\alpha\lambda$ towards zero and is clipped there. The readout counts weights that are exactly zero.

L1 $\wedge$

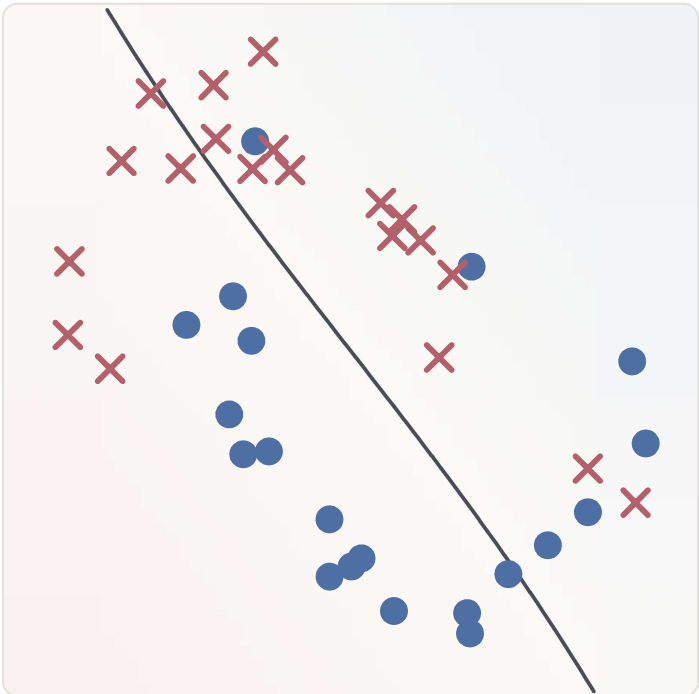
0	0.001	0.003	0.01	0.03
---	-------	-------	------	------

Train

Reset

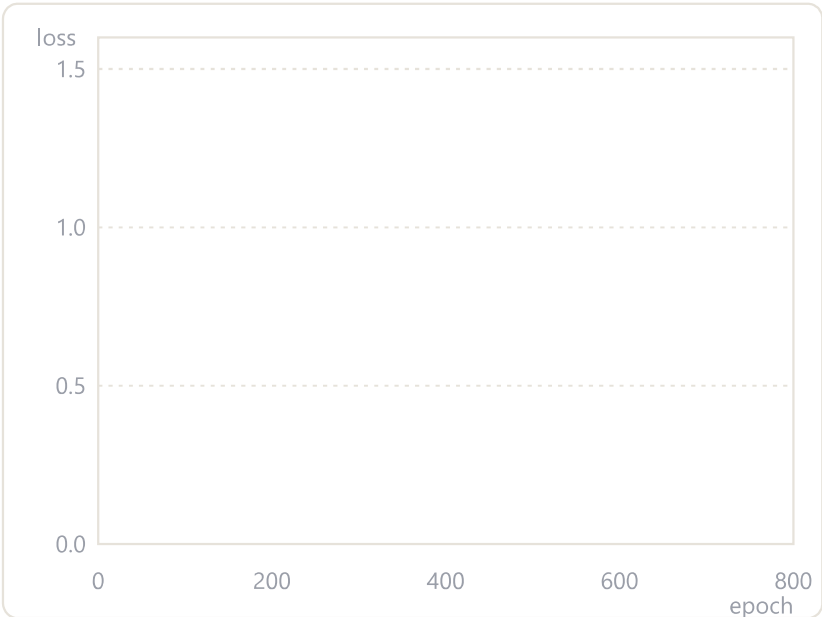
SPEED slow normal fast

DECISION BOUNDARY



● class ○ × class × □ validation points

LOSS PER EPOCH



— training loss — validation loss ○ lowest validation loss

EPOCH	TRAIN LOSS	TRAIN ACC	VAL LOSS	VAL ACC	LARGEST W	WEIGHTS AT 0
0	–	–	–	–	0.34	0%

WHAT TO NOTICE

- Even with 90% of the weights at exactly zero ($\lambda = 0.003$), the model still overfits. The few surviving weights are enough to build islands.
- L1 only regularises here at $\lambda = 0.03$, and the boundary it leaves is coarser than L2's.

WHY THE RESULT DIFFERS FROM L2

L2 at 0.03 keeps all 2 751 weights alive and small: a smooth boundary with no closed island, but it still bends towards three of the four mislabelled points, and validation loss creeps up to about 0.64. L1 at 0.03 keeps **ten** weights and sets the rest to zero: a tiny sub-network that can draw two arcs and nothing else, validation loss about 0.54.

Which is better here? L1, narrowly, because this problem needs very few weights. Two arcs is a job for a handful of neurons, and L1 found that handful. But look at how it got there: 0.01 zeroed 99% of the weights and **still overfit**, then 0.03 worked. That is the cliff of 3.6.

What would make L2 win? A finer search between 0.03 and 0.1, or a problem where many weights each carry a little signal, which is most real problems. There, zeroing weights throws useful signal away, and L2's gentle shrink keeps it.

PART 4

Early stopping

The simplest method of all: stop training before the model starts memorising.

4.1 What early stopping is

Intuitively: training has two phases. First the model learns the **pattern**, and both losses fall. Then it starts memorising the **noise**: training loss keeps falling, validation loss stops falling and turns up. Stop at the border between the two.

Formally: measure the validation loss after every epoch. While it is still going down, keep training. When it stops improving, or the two curves start to separate, stop and keep the weights from the epoch with the lowest validation loss.

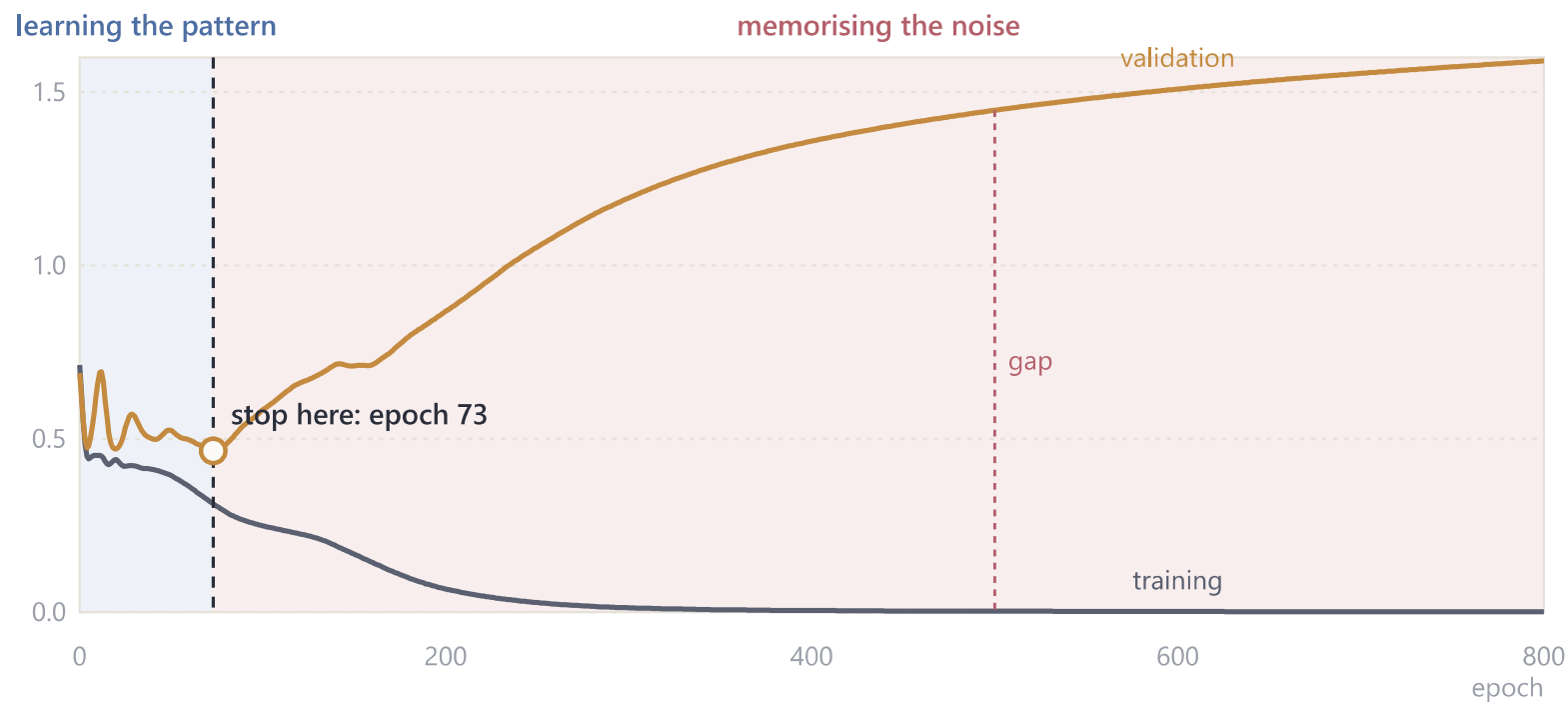


Figure 4.1 The unregularised run from Lab 1. Blue phase: both curves fall, the model is learning the pattern. Rose phase: the gap opens, the model is memorising noise. The border is epoch 73, the lowest validation loss.

4.2 Should we stop at the first bad epoch?

Validation loss is **noisy**. One epoch that is slightly worse than the last does not mean the model has started memorising. If we stop at the first rise, we often stop too soon.

So we wait. **Patience** is how many epochs in a row without a new best we allow before stopping. While we wait, we save the weights every time a new best appears.

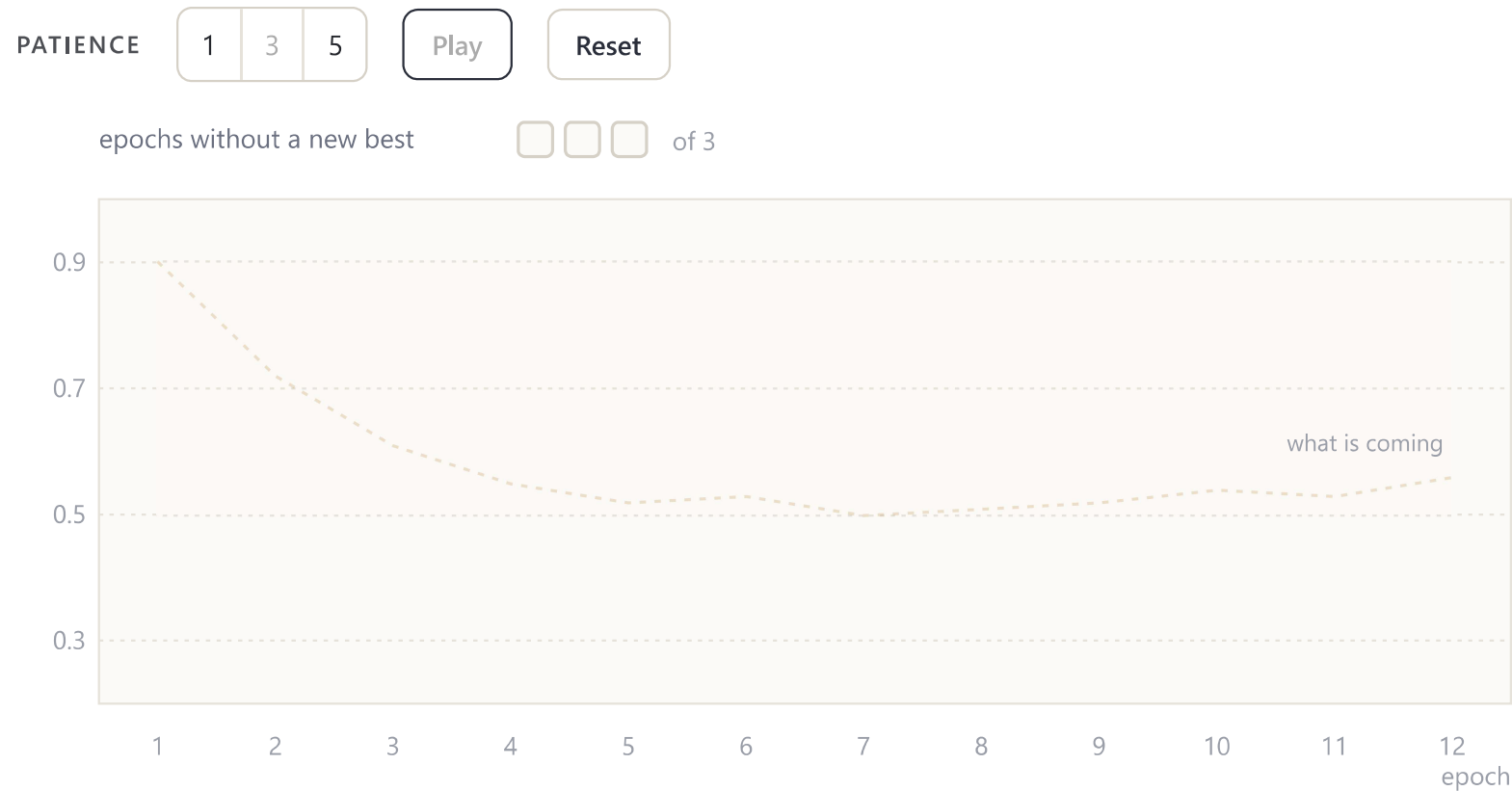


Figure 4.2 Twelve epochs of a noisy validation loss. Press Play and watch the counter: it rises on every epoch without a new best and resets when one appears. Training stops when the counter reaches the patience. The saved weights are the last new best.

- **Patience 1** stops after epoch 6 and restores epoch 5. It gave up too soon and missed epoch 7.
- **Patience 3** stops after epoch 10 and restores epoch 7, the true minimum.
- **Patience 5** finds the same epoch 7, at the cost of two more epochs of training.

4.3 Always restore the best weights

When training stops, the weights in the model are the **last** ones, not the best ones. With patience 3 above, that is epoch 10 (loss 0.54) rather than epoch 7 (loss 0.50). The saved weights have to be put back.

COMMON MISTAKE

Keras does not restore by default.

KERAS

```
EarlyStopping(monitor="val_loss",
              patience=20,
              restore_best_weights=True)
# default is False: keeps the last weights
```

PYTORCH (BY HAND)

```
if val_loss < best:
    best = val_loss
    best_state = copy.deepcopy(model.state_dict())
    wait = 0
else:
    wait += 1
    if wait >= patience: break
model.load_state_dict(best_state)
```

4.4 Why early weights are simple

- Weights start small, at initialisation, and grow gradually during training.
- Early on, the biggest drops in loss come from the **big, clear pattern**, which the model closes quickly.
- The **noise** is learned slowly, one point at a time, and needs many more epochs.

KEY IDEA

Stopping early keeps most of the pattern and little of the noise. Fewer steps also means smaller weights, so early stopping ends up doing a job very similar to L2.

4.5 Two notes

- Early stopping is almost free. Use it **nearly always**, as a safety net alongside the other methods.
- You chose the stopping epoch *using* the validation set, so that set has influenced the model. Keep a separate **test set** for the final score.

LAB 4 Stop at the right epoch

No penalty. The run stops once validation loss has not improved for the chosen number of epochs, then restores the best weights. The dashed line marks the best epoch so far.

EARLY STOPPING PATIENCE

off

5

20

50

☒ restore best weights

Train

Reset

SPEED

slow

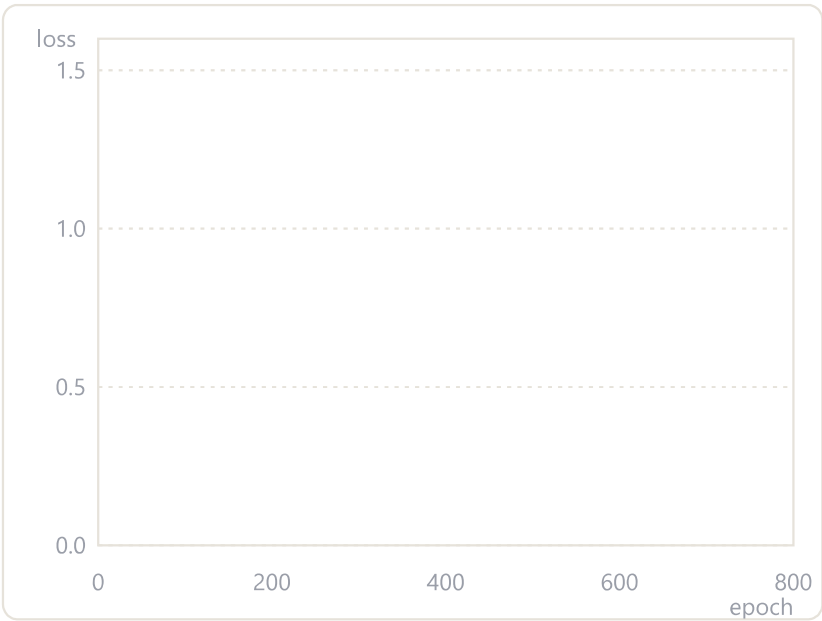
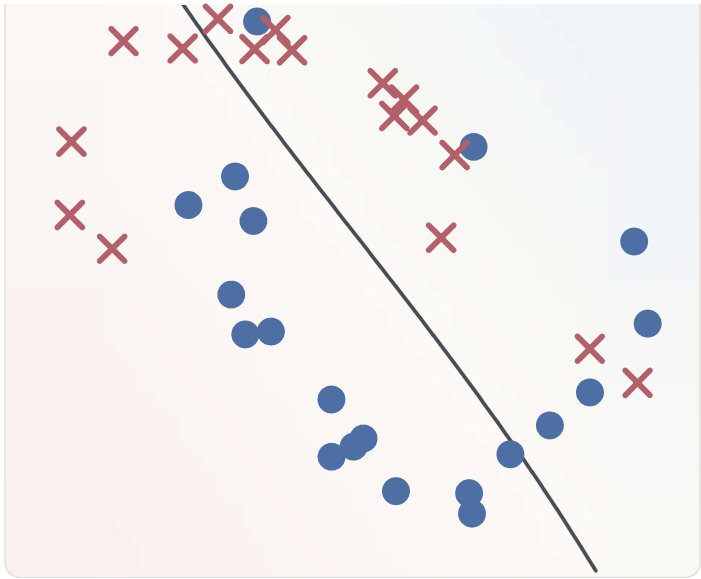
normal

fast

DECISION BOUNDARY

LOSS PER EPOCH





● class O ✕ class x □ validation points

— training loss — validation loss ○ lowest validation loss

EPOCH	TRAIN LOSS	TRAIN ACC	VAL LOSS	VAL ACC	LARGEST W
0	—	—	—	—	0.34

WHAT TO NOTICE

- The dashed line marks the best epoch so far. The run stops once it has waited *patience* epochs beyond it, and the boundary fades back to that epoch's shape.
- Patience 5 quits at epoch 9 with a nearly straight boundary. Patience 50 waits, finds epoch 73, and keeps the arcs without the islands.
- Switch *restore best weights* off and the run keeps the later, worse weights. Compare the validation loss.

PART 5

Dropout

Make memorising hard by switching neurons off at random.

5.1 Neurons that lean on each other

The finely balanced big weights behind an island (1.7) usually involve several neurons working together, each correcting another's quirks. This is called **co-adaptation**. In a team where any member may be absent on a given day, nobody can rely on one partner. Everyone has to understand the work themselves.

5.2 How dropout works

At each training step, every neuron in the layer is switched off with probability *r*, the dropout rate. A different random set is switched off every step.

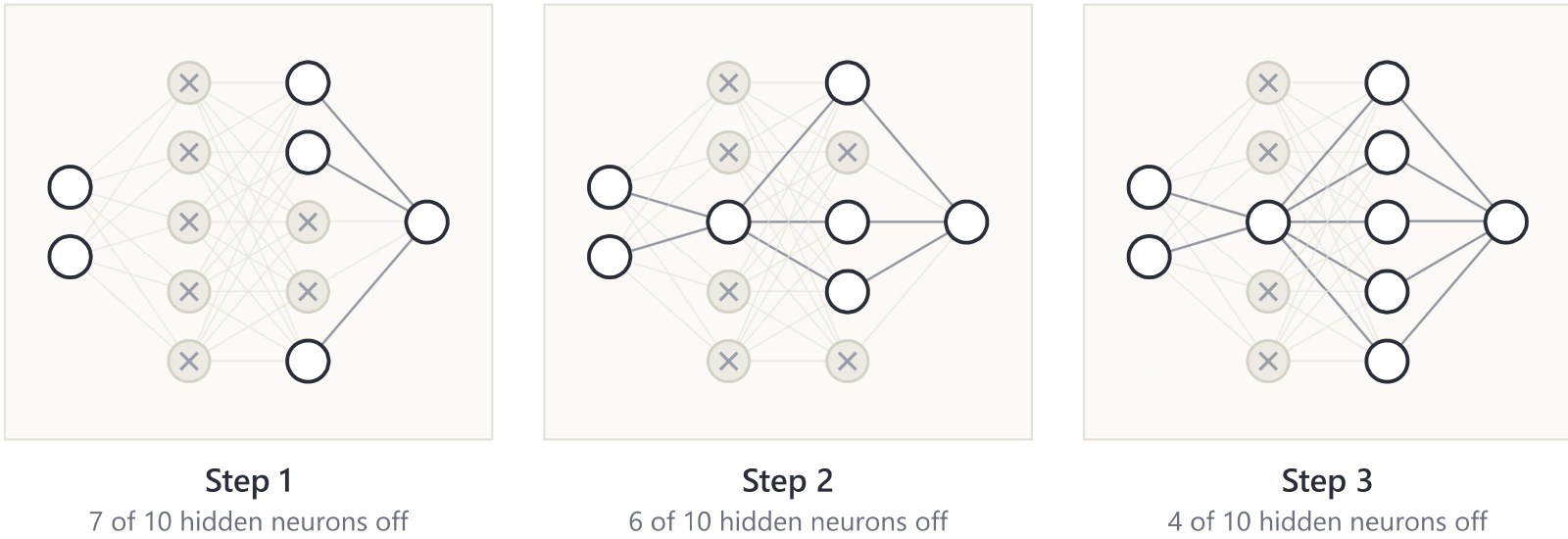


Figure 5.2 The same network over three training steps with $r = 0.5$. Greyed neurons are off for that step, and so are their connections.

5.3 The scaling problem

Four neurons each output 1.0 and feed a sum. With $r = 0.5$, about half are off during training, so the next layer receives about 2. At test time all four are on and it receives 4: **twice what it was trained on.**

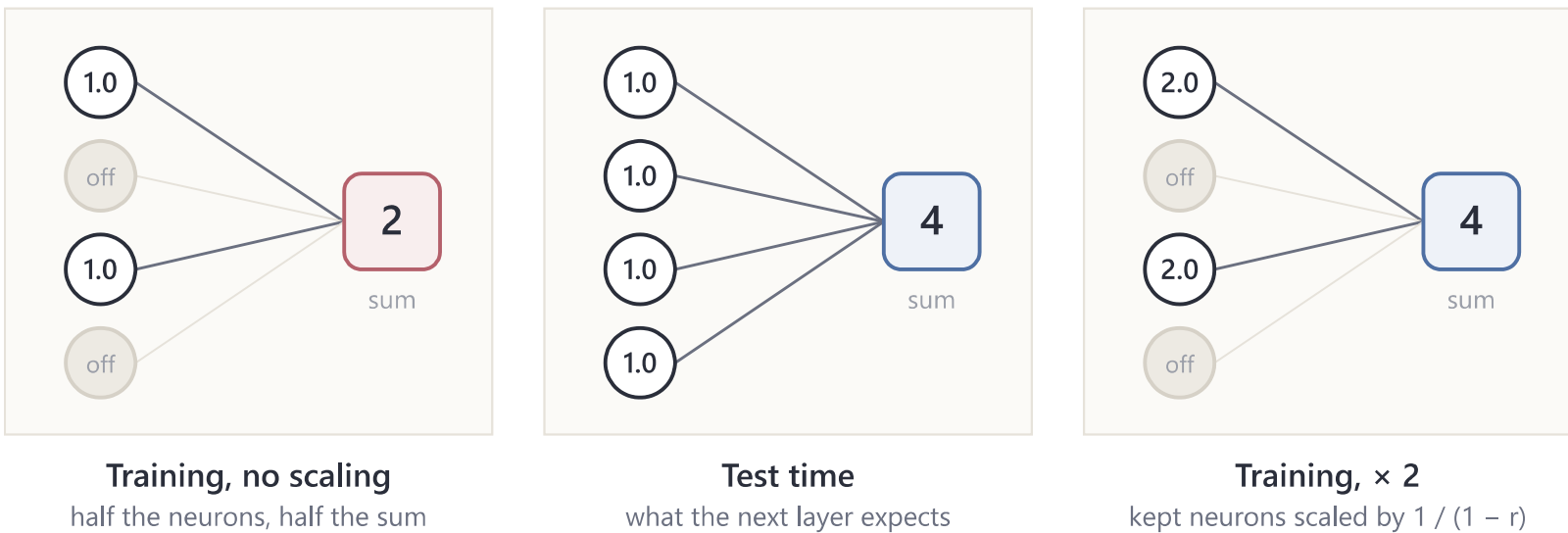


Figure 5.3 Left: training without scaling, the sum is 2. Middle: test time, all neurons on, the sum is 4. Right: **inverted dropout**, every kept neuron is multiplied by $1 / (1 - r) = 2$ during training, so the sum is 4 again and matches test time.

KEY IDEA

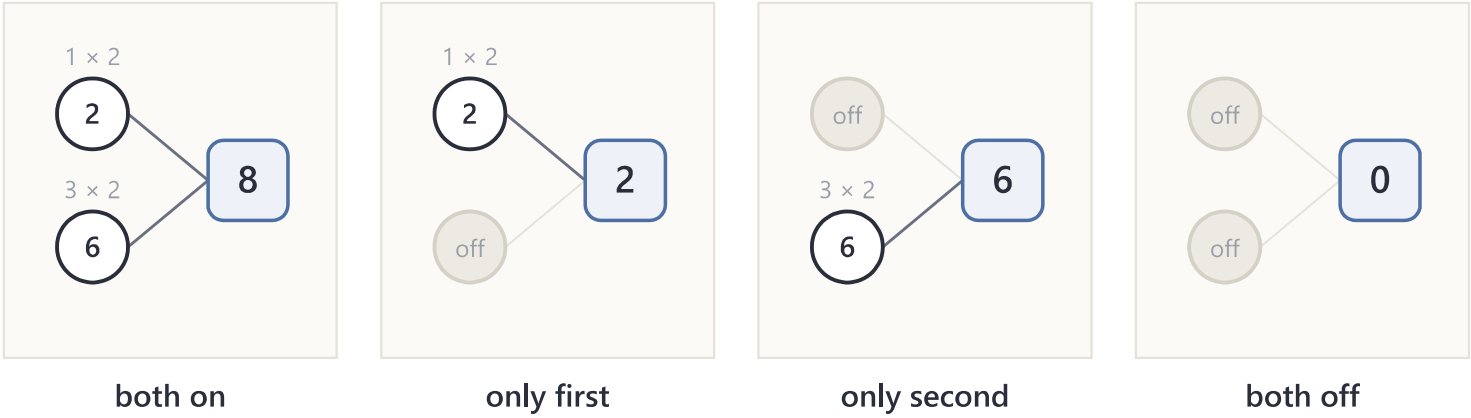
During training, multiply every kept neuron by $1 / (1 - r)$. At test time change nothing: all neurons on, no scaling. One step is noisy, but **on average** training matches testing.

5.4 Why it works (1): no neuron can lean on a partner

Any partner might vanish on the next step, so each neuron must learn something that is **useful by itself**. Fragile, finely balanced combinations, the building blocks of islands, never become reliable, and the model stops depending on them.

5.5 Why it works (2): many networks in one

Every dropout mask gives a different smaller network, and all of them share the same weights. Three hidden neurons give $2^3 = 8$ possible networks; a hundred give 2^{100} . Averaging many different models cancels their individual mistakes, an **ensemble**, and dropout trains a huge one at the cost of a single network.



average of the four outputs: $(8 + 2 + 6 + 0) / 4 = 4 =$ the full network, $1 + 3$, at test time

Figure 5.5 Two neurons contributing 1 and 3 to an output, $r = 0.5$, scale 2. The four possible masks give 8, 2, 6 and 0. Their average is 4, exactly what the full network gives at test time. Exact for a linear layer, approximate once there is an activation.

5.6 Training mode and test mode

Dropout is **on only during training**.

COMMON MISTAKE

PYTORCH

```
model.train() # dropout on
...
model.eval()  # dropout off: do not forget
with torch.no_grad():
    preds = model(x_test)
```

KERAS

```
model.fit(...) # dropout on
model.predict(...) # dropout off, automatic

# Dropout(0.5) in the model definition
```

5.7 Costs and surprises

- **Slower, noisier training.** You usually need more epochs.
- **Small network + dropout = underfitting.** A small network cannot afford to lose neurons. Add dropout to networks big enough to overfit.
- **Typical rates:** 0.2 to 0.5 on hidden layers, 0 to 0.2 on inputs.
- **Surprise:** with dropout, training loss can sit **above** validation loss. Training loss is measured *with neurons switched off*, validation uses the full network.

LAB 5 Switch neurons off

No penalty. Each training step switches every hidden neuron off with probability r , and scales the kept ones as in 5.3. Compare the wide network with a narrow one.

DROPOUT RATE R				HIDDEN WIDTH	
0	0.2	0.5	0.7	4 · 4	50 · 50



4. Re-read the curves and the gap after every change.
5. Report the **test** score **once**, at the end.

6.3 Stacking has diminishing returns

Every method attacks the same gap. Once the gap is closed, adding another method only pushes the model into **underfitting**.

6.4 One-page comparison

	L2	L1	Early stopping	Dropout
What it does	Shrinks all weights in proportion	Subtracts a fixed amount; some weights become 0	Stops before the noise is learned	Switches neurons off at random
Boundary	Smooth	Smooth, ignores useless inputs	An earlier, simpler epoch	Smooth, robust
Your dial	λ	λ	patience	rate r
Watch out	Use AdamW; scale features	Rarely helps hidden layers	Restore best weights	Needs more epochs; hurts small networks

6.5 Other tools you will meet later

- **More data:** the best regulariser of all.
- **Data augmentation:** make new training examples (flip or crop an image).
- **Batch normalisation:** mainly speeds up training, with a mild regularising side effect.
- **Label smoothing:** stop the model being 100% sure of its answers.

LAB 6 Tune like a practitioner

Every control at once, plus a locked test set of 200 points. Tune with the validation readout. Press the test button once, at the end.

HIDDEN WIDTH

4 · 4

50 · 50

L2 λ

0

0.003

0.01

0.03

0.1

0.3

L1 λ

0

0.001

0.003

0.01

0.03

DROPOUT RATE R

0

0.2

0.5

0.7

EARLY STOPPING PATIENCE

off

5

20

50

☒ restore best weights

Train

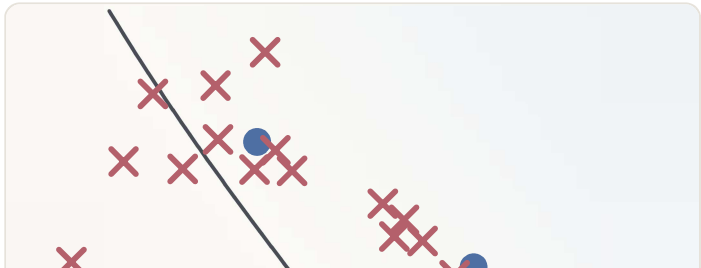
Reset

SPEED

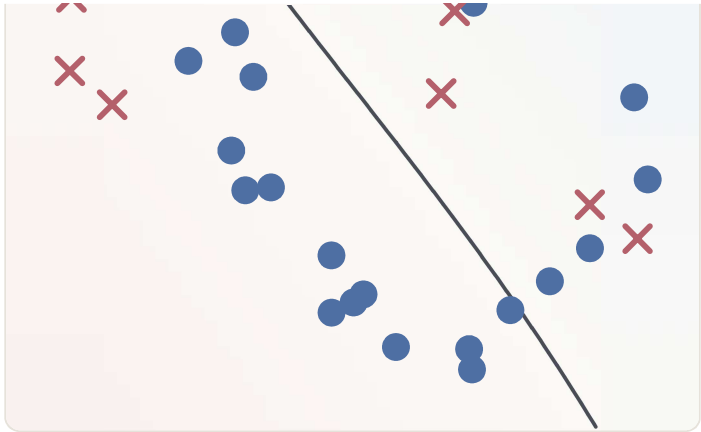
slow

normal

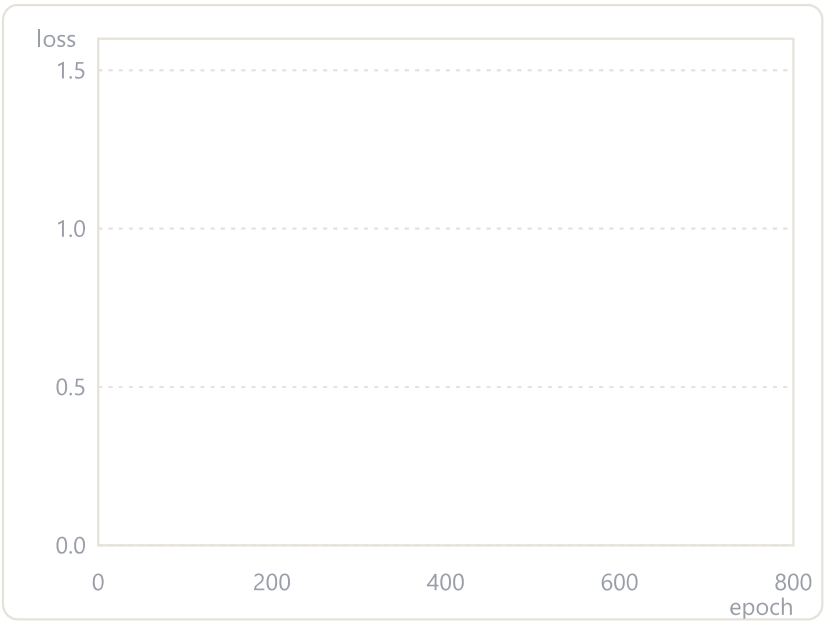
fast

DECISION BOUNDARY

LOSS PER EPOCH



● class ○ × class × □ validation points



— training loss — validation loss ○ lowest validation loss

EPOCH	TRAIN LOSS	TRAIN ACC	VAL LOSS	VAL ACC	LARGEST W	WEIGHTS AT 0
0	—	—	—	—	0.34	0%

Evaluate on the test set

Each press is a look at the exam paper.

TRY IT: THE RECIPE

Fill the table from Lab 6. Use the validation numbers to choose; press the test button only for the final row.

Setting	Train acc	Val loss	Val acc
Nothing	_____	_____	_____
Patience 50 only	_____	_____	_____
Dropout 0.5 only	_____	_____	_____
L2 0.03 + dropout 0.5	_____	_____	_____
Your best (test once)	_____	_____	_____

Show answer

Roughly: nothing 100 / 1.59 / 80. Patience 50: 88 / 0.47 / 85. Dropout 0.5: 88 / 0.78 / 87. L2 0.03 + dropout 0.5: 80 / 0.53 / 77, which is underfitting: two regularisers stacked have pushed the model too far. One well-chosen method beats two.

PART 7

Check yourself

Eight questions and three short explanations.

MULTIPLE CHOICE

1. Training accuracy 99%, validation 71%, validation loss rising since epoch 30. Best first move?

- (a) Train more epochs (b) Add early stopping or L2 (c) Make the network larger (d) Raise the learning rate
2. After raising λ from 0.001 to 1, training accuracy is 70% and validation 69%. What happened?
- (a) Overfitting (b) Underfitting, so lower λ (c) Healthy (d) The data is broken
3. Which method can make a weight **exactly** zero?
- (a) L2 (b) L1 (c) Dropout (d) Early stopping
4. Dropout rate 0.25. A kept neuron outputs 2.0. During training it becomes
- (a) 1.5 (b) 2.0 (c) 2.67 (d) 8.0
5. With dropout, training loss sits above validation loss. Why?
- (a) A bug (b) Training loss is measured with neurons switched off (c) Validation data is easier (d) Learning rate too high
6. Why use AdamW instead of Adam + L2?
- (a) It is faster (b) Adam rescales the penalty along with the gradient, so equal weights shrink unequally (c) L2 does not work with Adam (d) AdamW has no learning rate
7. Patience 2, validation losses 0.80, 0.60, 0.62, 0.59, 0.61, 0.63. When does training stop, and which epoch is restored?
8. Why do we not penalise biases?

Show answers

1 (b). 2 (b). 3 (b). 4 (c): 2.0 / 0.75. 5 (b). 6 (b). 7: stops after epoch 6, restores epoch 4. 8: a bias only shifts the boundary; it cannot create bends or islands, and there are very few of them.

EXPLAIN IN TWO OR THREE SENTENCES

1. Why do small weights produce smooth decision boundaries?
2. Why does L1 reach zero when L2 does not?
3. Why does stopping early act like a penalty on weights?

Show model answers

1. A neuron's output changes by about w per unit of input. With small w nearby inputs get nearly the same output, so the boundary cannot bend sharply or wrap a single point (1.7).
2. L2's push is λw and fades as w shrinks, so it approaches zero without arriving. L1's push is a constant λ that does not fade, so it carries the weight all the way (3.3, 3.4).
3. Weights start small and grow with training. Fewer steps means less growth, and the clear pattern is learned in the first steps while the noise needs many more (4.4).

Worksheet 09 of the Deep Learning interactive worksheets. Everything runs in your browser; nothing is loaded from the network. [All worksheets.](#)